



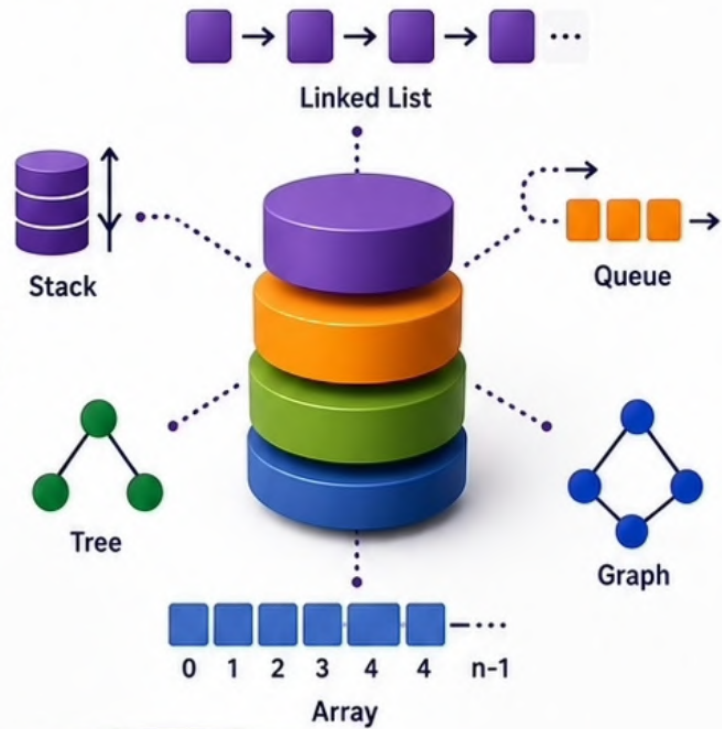
AS PER
LATEST
SYLLABUS

MCA-203

Data Structures

ORGANIZE • OPTIMIZE • PERFORM

By Virendra Goura



Stack Implementation

```
class Stack:
    def __init__(self):
        self.items = []

    def push(self, item):
        self.items.append(item)

    def pop(self):
        if not self.items:
            return "Stack Underflow"
        return self.items.pop()

    def is_empty(self):
        return len(self.items) == 0
```

Build Logic. Shape Solutions.



EASY TO
UNDERSTAND



PRACTICAL
EXAMPLES



CONCEPTS
MADE SIMPLE



BOOST YOUR
PROBLEM SOLVING



www.virendragoura.com

PREFACE

Dear Reader,

It is a privilege to present this comprehensive collection of RTU MCA Semester Examination Notes, designed to cover the complete syllabus with clarity and academic accuracy. Every concept, definition, explanation, and example has been organized to support thorough understanding and effective exam preparation.

The purpose of these notes is to simplify complex topics and provide a reliable study resource that can assist in both detailed learning and quick revision. Continuous effort has been made to ensure correctness and relevance; however, learning grows when readers engage, question, and explore further.

May these notes serve as a strong academic foundation and contribute meaningfully to your preparation and future growth.

Warm regards,
Virendra Goura
www.virendragoura.com

DISCLAIMER

This e-book has been created with utmost care, sincere effort, and extensive proofreading. However, despite all attempts to avoid mistakes, there may still be some errors, omissions, or inaccuracies that remain unnoticed.

This e-book is issued with the understanding that neither the author nor the publisher shall be held responsible for any loss, damage, or misunderstanding arising from the use of the information contained within.

All content provided is for educational and informational purposes only.

© 2026— All Rights Reserved.

No part of this e-book may be reproduced, copied, scanned, stored in a retrieval system, or transmitted in any form—whether electronic, mechanical, digital, or otherwise—without prior written permission from the author/publisher.

Any unauthorized use, sharing, or distribution of the material is strictly prohibited and may lead to civil or criminal liability under applicable copyright law.

MCA - 203 Data Structures

Unit-1 Introduction & Stack

Basic data structures such as arrays, linked list, stack, trees and queues and their applications, linked and sequential representation Basic Terminology, Elementary Data organization, Data Structure operations. Preliminaries of algorithm, Algorithm analysis and complexity.

Stack: Implementation of stack, operations on stack. Applications of stack: Conversion of in fix-expressions to prefix and postfix expressions, evaluation of postfix expression.

Unit-2 Queues & Linked List

Queues: Implementation of queues, Operations on Queue, Types of Queues - Circular queue, Dequeue and Priority Queue.

Linked List: Representation and Implementation of Singly Linked Lists, Two-way Header List, Traversing and Searching of Linked List, insertion and deletion to/from Linked Lists, insertion and deletion Algorithms, Doubly linked list, Header lists, circular lists, sorted lists.

Unit-3 Trees

Basic terminology and definitions. Array and Linked Representation of Binary trees, Traversing Binary trees. Binary Search Trees: Binary Search Tree (BST), Traversal, Insertion and Deletion in BST, and Introduction to balanced BST (AVLTrees).

Unit-4 Searching & Sorting

Searching: Sequential search, binary search, comparison and analysis.

Sorting: Insertion Sort, Bubble Sort, Quick Sort, Two-Way-Merge Sort, Heap Sort, Sorting on Different Keys, Practical consideration for internal Sorting.

Unit-5 Graphs

Terminology & Representations, Graphs & Multi-graphs, Directed Graphs, Sequential Representations of Graphs, Adjacency Matrices, Traversal, Connected Component and Spanning Trees, Minimum Cost Spanning Trees.

Unit-1 Introduction & Stack

Basic data structures such as arrays, linked list, stack, trees and queues and their applications, linked and sequential representation Basic Terminology, Elementary Data organization, Data Structure operations. Preliminaries of algorithm, Algorithm analysis and complexity.

Stack: Implementation of stack, operations on stack. Applications of stack: Conversion of in fix-expressions to prefix and postfix expressions, evaluation of postfix expression.

Introduction to Data Structures

A data structure is a systematic method of organizing, storing, and managing data inside computer memory so that data can be processed efficiently. Data structures are very important in computer science because they improve execution speed, optimize memory usage, and simplify implementation of algorithms. Efficient data structures reduce computational complexity and increase software performance in real-world applications.

Data structures are widely used in:

- Operating Systems
- Database Management Systems
- Computer Networks
- Artificial Intelligence
- Compiler Design
- Web Applications
- Software Engineering

Efficient data structures help developers design scalable, fast, and reliable software systems.

Characteristics of Data Structures

Data structures provide organized methods for storing and processing information in computer systems. They improve memory utilization, increase processing speed, and allow efficient implementation of algorithms. Good data structures reduce computational complexity and help programmers manage large volumes of information efficiently in modern software and communication systems.

Main Characteristics

- Efficient data organization
- Faster data processing
- Better memory management
- Improved scalability
- Easy implementation of algorithms

Classification of Data Structures

Data structures are mainly classified into linear and non-linear data structures depending upon arrangement of elements and relationships between data items. Understanding classification helps programmers select suitable structures according to computational and software requirements.

Type	Description
Linear Data Structure	Elements arranged sequentially
Non-Linear Data Structure	Elements arranged hierarchically

Linear Data Structures

Linear data structures store elements in sequential order where each element is connected logically to previous and next element. Traversal occurs in single direction. These structures are simple to implement and widely used in software systems and programming applications.

Examples

- Arrays
- Linked Lists
- Stacks
- Queues

Non-Linear Data Structures

Non-linear data structures organize data hierarchically or interconnected in multiple directions. These structures are useful for representing complex relationships and hierarchical information in computer systems, databases, networking, and artificial intelligence applications.

Examples

- Trees
- Graphs

Arrays

An array is collection of elements of same data type stored in contiguous memory locations. Arrays allow direct access of elements using index numbers, making data retrieval very fast. Arrays are one of the simplest and most commonly used data structures in programming and computational systems.

Features of Arrays

Arrays provide indexed access and sequential memory allocation, making processing operations fast and efficient. Arrays are suitable for storing homogeneous data and implementing searching and sorting algorithms. However, arrays have fixed size limitation and require contiguous memory allocation.

Features

- Fixed size structure
- Indexed element access
- Sequential memory allocation
- Homogeneous data storage

Array Diagram

Index :	0	1	2	3	4
Data :	10	20	30	40	50

Advantages of Arrays

Arrays provide fast access to elements because indexing allows direct retrieval without traversal. They are simple to implement and efficient for processing fixed-size data collections. Arrays are suitable for mathematical operations, sorting algorithms, and matrix representations where continuous memory allocation improves processing speed.

Disadvantages of Arrays

Arrays have fixed size, meaning memory cannot be increased dynamically during execution. Insertion and deletion operations are costly because elements may need shifting. Arrays also require contiguous memory allocation, which may create memory wastage in dynamic applications.

Applications of Arrays

Arrays are widely used in matrix operations, searching algorithms, sorting techniques, image processing systems, database indexing, operating systems, and scientific computing applications because of their fast access speed and simple implementation structure.

Algorithm for Array Traversal

Traversal means visiting each element of array exactly once for processing or displaying data. Traversal operation forms basis of searching, sorting, and data manipulation algorithms in programming systems.

Step 1: Start
Step 2: Read array elements
Step 3: Initialize $i = 0$
Step 4: Repeat until $i < n$
Step 5: Print $arr[i]$
Step 6: Increment i
Step 7: Stop

C Program for Array Traversal

```
#include <stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};
    int i;

    printf("Array Elements:\n");

    for(i=0; i<5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}
```

Output

Array Elements:
10 20 30 40 50

Linked List

A linked list is dynamic linear data structure consisting of nodes connected through pointers. Each node contains data and address of next node. Unlike arrays, linked lists do not require contiguous memory allocation and support dynamic memory management.

Structure of Node

[Data | Next Address]

Features of Linked Lists

Linked lists provide dynamic memory allocation and flexible data storage because nodes can be added or removed during runtime. They improve insertion and deletion efficiency but require additional memory for storing pointer information.

Features

- Dynamic size
- Pointer-based structure
- Non-contiguous memory allocation
- Efficient insertion and deletion

Types of Linked Lists

Type	Description
Singly Linked List	One pointer
Doubly Linked List	Two pointers
Circular Linked List	Last node connects to first

Linked List Diagram



Advantages of Linked Lists

Linked lists provide dynamic memory allocation and flexible size management. Insertion and deletion operations are efficient because no shifting of elements is required. Linked lists improve memory utilization because memory is allocated only when required.

Disadvantages of Linked Lists

Linked lists require additional memory for storing pointers. Accessing elements is slower compared to arrays because traversal must occur sequentially. Pointer handling increases complexity of implementation and debugging.

Applications of Linked Lists

Linked lists are used in browser history management, dynamic memory allocation, music playlists, polynomial representation, operating system scheduling, and implementation of stacks and queues.

C Program for Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node *head, *second;

    head = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = NULL;

    printf("%d %d", head->data, second->data);

    return 0;
}
```

Output

10 20

Stack

A stack is linear data structure following: LIFO (Last In First Out) principle. In stack, insertion and deletion operations occur only at one end called TOP. The last inserted element is removed first. Stacks are widely used in recursion handling, expression evaluation, compiler design, and memory management systems.

Real Life Examples of Stack

Stacks can be observed in daily life situations such as stack of plates, browser back button functionality, undo operations in editors, and recursive function calls in programming systems. These examples help understand working principle of stack data structure.

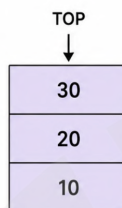
Basic Operations of Stack

Stack operations manage insertion, deletion, and access of elements efficiently. These operations are essential for implementing stack-based algorithms and applications.

Stack Operations

- Push
- Pop
- Peek
- isEmpty
- isFull

Stack Diagram



Push Operation

Push operation inserts new element into stack at TOP position. Before insertion, stack checks overflow condition. After insertion, TOP value increases and new element becomes topmost element of stack.

Pop Operation

Pop operation removes topmost element from stack. Before deletion, stack checks underflow condition. After removal, TOP value decreases and next lower element becomes topmost element.

Peek Operation

Peek operation displays topmost element without removing it from stack. It is useful for expression evaluation and stack inspection operations.

Stack Implementation Using Array

Array implementation stores stack elements in contiguous memory locations. TOP variable keeps track of topmost element. This implementation is simple and efficient but has fixed size limitation.

Stack Representation

```
stack[5]  
TOP = -1
```

PUSH Algorithm

```
Step 1: Start  
Step 2: Check if TOP = MAX-1  
Step 3: If true print Overflow  
Step 4: Else TOP = TOP + 1  
Step 5: Insert element at stack[TOP]  
Step 6: Stop
```

POP Algorithm

Step 1: Start
Step 2: Check if TOP = -1
Step 3: If true print Underflow
Step 4: Else remove stack[TOP]
Step 5: TOP = TOP - 1
Step 6: Stop

C Program for Stack Using Array

```
#include <stdio.h>

#define MAX 5

int stack[MAX];
int top = -1;

void push(int value)
{
    if(top == MAX-1)
    {
        printf("Stack Overflow\n");
    }
    else
    {
        top++;
        stack[top] = value;
    }
}

void pop()
{
    if(top == -1)
    {
        printf("Stack Underflow\n");
    }
    else
    {
        printf("Deleted Element = %d\n", stack[top]);
        top--;
    }
}

void display()
{
    int i;

    for(i=top; i>=0; i--)
    {
        printf("%d\n", stack[i]);
    }
}

int main()
{
    push(10);
    push(20);
    push(30);

    display();
}
```

```

    pop();

    return 0;
}

```

Output

```

30
20
10
Deleted Element = 30

```

Stack Implementation Using Linked List

Linked list implementation uses dynamically allocated nodes connected through pointers. This implementation removes fixed size limitation and improves memory utilization.

Linked Stack Diagram



C Program for Stack Using Linked List

```

#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

struct node *top = NULL;

void push(int value)
{
    struct node *newnode;

    newnode = (struct node*)malloc(sizeof(struct node));

    newnode->data = value;
    newnode->next = top;
    top = newnode;
}

void display()
{
    struct node *temp = top;

    while(temp != NULL)
    {
        printf("%d\n", temp->data);
        temp = temp->next;
    }
}

int main()
{
    push(10);
}

```

```

push(20);
push(30);

display();

return 0;
}

```

Output

```

30
20
10

```

Queue

A queue is linear data structure following:

FIFO (First In First Out)

principle. Insertion occurs at rear end and deletion occurs at front end. Queues are widely used in scheduling systems and networking applications.

Queue Diagram



Applications of Queue

Queues are used in CPU scheduling, printer management, networking packet processing, and ticket reservation systems where first arriving element must be processed first.

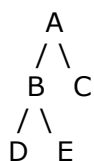
Trees

A tree is hierarchical non-linear data structure consisting of nodes connected through edges. Trees are used for representing hierarchical relationships and improve searching efficiency in database systems.

Tree Terminology

Term	Meaning
Root Node	Topmost node
Parent Node	Upper connected node
Child Node	Lower connected node
Leaf Node	Node without children
Edge	Connection between nodes

Tree Diagram



Applications of Trees

Trees are used in database indexing, file systems, compiler syntax analysis, XML parsing, and artificial intelligence systems.

Linked Representation

Linked representation stores elements dynamically using pointers and nodes. Memory allocation occurs during runtime, improving flexibility and insertion/deletion efficiency.

Sequential Representation

Sequential representation stores elements in contiguous memory locations. Arrays commonly use sequential representation because they support direct indexing access.

Basic Terminology

Term	Meaning
Data	Raw facts
Node	Individual element
Pointer	Stores address
Overflow	Structure full
Underflow	Structure empty

Elementary Data Organization

Elementary Data Organization refers to the systematic arrangement of data from the smallest unit to larger storage structures in a computer system.

It helps in:

- Storing data properly
- Easy data access
- Efficient data processing
- Better data management

Levels of Organization

Bit → Byte → Field → Record → File

Data Structure Operations

Data structures support several operations for efficient data processing and management.

Operations

- Traversal
- Insertion
- Deletion
- Searching

- Sorting

Traversal

Traversal means visiting every element exactly once for processing or displaying information.

Insertion

Insertion operation adds new element into data structure at specified location.

Deletion

Deletion operation removes existing element while maintaining structural consistency.

Searching

Searching operation finds required element using algorithms such as linear search and binary search.

Sorting

Sorting operation arranges data elements in ascending or descending order for efficient processing.

Preliminaries of Algorithm

An algorithm is finite sequence of clearly defined instructions used to solve computational problems systematically. Algorithms form foundation of software development and programming systems.

Characteristics of Algorithm

Characteristic	Explanation
Finite	Limited steps
Definite	Clear instructions
Input	Accepts data
Output	Produces result
Effective	Executable steps

Example of Algorithm

Step 1: Start
Step 2: Read A and B
Step 3: Sum = A + B
Step 4: Print Sum
Step 5: Stop

Algorithm Analysis and Complexity

Algorithm analysis is the process of evaluating the performance of an algorithm. It helps us understand:

- How much **time** an algorithm takes

- How much **memory (space)** it uses
- Which algorithm is more efficient

Complexity means the amount of resources required by an algorithm.

Need of Algorithm Analysis

- To compare different algorithms
- To reduce execution time
- To save memory
- To improve system performance
- To select the best solution for a problem

Types of Complexity

(A) Time Complexity

Time complexity measures the amount of time required by an algorithm to execute.

Example in C

```
#include <stdio.h>

int main() {
    int i;
    for(i = 0; i < 5; i++) {
        printf("%d\n", i);
    }
    return 0;
}
```

Here loop runs 5 times.

Time Complexity:

$O(n)$

(B) Space Complexity

Space complexity measures the amount of memory used by an algorithm.

Example

```
int a[100];
```

Memory is used to store 100 integers.

Asymptotic Notations

Asymptotic notations are used to represent algorithm complexity.

Main types:

1. Big O Notation

2. Omega Notation
3. Theta Notation

Big O Notation – O()

Big O represents the **worst-case complexity**.

It shows maximum time taken by an algorithm.

Common Big O Complexities

Complexity	Meaning
O(1)	Constant Time
O(log n)	Logarithmic Time
O(n)	Linear Time
O(n log n)	Linearithmic
O(n ²)	Quadratic
O(2 ⁿ)	Exponential

Example of O(1)

```
#include <stdio.h>
```

```
int main() {  
    int a = 10;  
    printf("%d", a);  
    return 0;  
}
```

Execution time remains constant.

Complexity:

O(1)

Example of O(n)

```
#include <stdio.h>
```

```
int main() {  
    int i;  
    for(i = 0; i < n; i++) {  
        printf("%d", i);  
    }  
}
```

Loop runs n times.

Complexity:

O(n)

Example of O(n²)

```
#include <stdio.h>

int main() {
    int i, j;

    for(i = 0; i < n; i++) {
        for(j = 0; j < n; j++) {
            printf("*");
        }
    }
}
```

Nested loops execute $n \times n$ times.

Complexity:

$O(n^2)$

Omega Notation – $\Omega()$

Omega notation represents the **best-case complexity**.

It shows minimum time required.

Example

Linear search when element is found at first position.

Complexity:

$\Omega(1)$

Theta Notation – $\Theta()$

Theta notation represents the **average-case complexity**.

It gives tight bound.

Example

$\Theta(n)$

Best Case, Worst Case, Average Case

Case	Meaning
Best Case	Minimum execution time
Worst Case	Maximum execution time
Average Case	Average execution time

Example: Linear Search

Best Case

Element found at first position.

$O(1)$

Worst Case

Element found at last position.

$O(n)$

Average Case

Element found in middle.

$O(n)$

Complexity of Common Operations

Operation	Complexity
Array Access	$O(1)$
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Bubble Sort	$O(n^2)$
Insertion Sort	$O(n^2)$
Merge Sort	$O(n \log n)$
Quick Sort	$O(n \log n)$

Linear Search Program in C

```
#include <stdio.h>
```

```
int main() {  
    int a[5] = {10, 20, 30, 40, 50};  
    int i, key = 30;  
  
    for(i = 0; i < 5; i++) {  
        if(a[i] == key) {  
            printf("Element Found");  
            break;  
        }  
    }  
  
    return 0;  
}
```

Complexity

Best Case:

$O(1)$

Worst Case:

$O(n)$

Applications of Stack

Stacks are used in recursion handling, compiler design, expression conversion, expression evaluation, syntax parsing, browser navigation, and backtracking algorithms.

Infix Expression

In infix expression, operator appears between operands.

Example

A + B

Prefix Expression

In prefix expression, operator appears before operands.

Example

+AB

Postfix Expression

In postfix expression, operator appears after operands.

Example

AB+

Conversion of Infix to Postfix

Stack is used for temporarily storing operators according to precedence rules during conversion process.

Example

Infix:

A + B * C

Postfix:

ABC*+

C Program for Infix to Postfix

```
#include <stdio.h>
```

```
int main()
{
    char infix[] = "A+B*C";
    char postfix[] = "ABC*+";

    printf("Infix Expression = %s\n", infix);
    printf("Postfix Expression = %s\n", postfix);

    return 0;
}
```

Output

Infix Expression = A+B*C
Postfix Expression = ABC*+

Conversion of Infix to Prefix

Infix expressions can also be converted into prefix notation using stack operations.

Example

Infix:

A + B

Prefix:

+AB

Evaluation of Postfix Expression

Postfix expressions are evaluated using stack. Operands are pushed into stack while operators perform operations on top elements.

Example

23*54*+

Final Answer:

26

C Program for Postfix Evaluation

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int result;
```

```
    result = (2 * 3) + (5 * 4);
```

```
    printf("Result = %d", result);
```

```
    return 0;
```

```
}
```

Output

Result = 26

Unit-2 Queues & Linked List

Queues: Implementation of queues, Operations on Queue, Types of Queues - Circular queue, Dequeue and Priority Queue.

Linked List: Representation and Implementation of Singly Linked Lists, Two-way Header List, Traversing and Searching of Linked List, insertion and deletion to/from Linked Lists, insertion and deletion Algorithms, Doubly linked list, Header lists, circular lists, sorted lists.

Introduction to Queue

A queue is linear data structure that follows: FIFO (First In First Out)

principle. In queue, insertion operation is performed at rear end while deletion operation is performed at front end. The first inserted element is removed first. Queues are widely used in operating systems, networking systems, printer scheduling, CPU scheduling, and communication systems where fair sequential processing is required.

Queues are important because they maintain proper order of processing and improve efficiency of resource management in computer systems and software applications.

Real Life Examples of Queue

Queues can be observed in many real-life situations such as ticket reservation counters, ATM lines, printer queues, customer service systems, and CPU scheduling. In all these systems, the first arriving person or process gets serviced first, following FIFO principle.

Basic Operations of Queue

Queues support several operations used for insertion, deletion, and accessing data elements efficiently. These operations help programmers manage queue structures and process sequential data correctly during program execution.

Queue Operations

- Enqueue
- Dequeue
- Peek/Front
- isEmpty
- isFull

Queue Representation



Enqueue Operation

Enqueue operation inserts new element at rear end of queue. Before insertion, system checks whether queue is full to avoid overflow condition. After insertion, rear pointer moves to next position and inserted element becomes last element of queue.

Dequeue Operation

Dequeue operation removes element from front end of queue. Before deletion, queue checks underflow condition to ensure queue is not empty. After deletion, front pointer moves forward to next available element in queue structure.

Advantages of Queue

Queues provide organized processing order and fair scheduling of tasks in computer systems. They are simple to implement and highly useful in operating systems, networking systems, and process management applications requiring sequential execution order.

Disadvantages of Queue

Queues provide access only from front and rear ends, limiting random access operations. Fixed-size queue implementations may cause overflow conditions, while empty queues may produce underflow conditions during deletion operations.

Applications of Queue

Queues are widely used in:

- CPU Scheduling
- Printer Management
- Networking Packet Processing
- Ticket Reservation Systems
- Keyboard Buffer Handling
- Call Center Systems

Queue Implementation Using Array

Array implementation stores queue elements in contiguous memory locations. Two variables called FRONT and REAR are used to manage insertion and deletion operations.

Queue Representation Using Array

```
queue[5]  
FRONT = -1  
REAR = -1
```

Algorithm for ENQUEUE Operation

```
Step 1: Start  
Step 2: Check if REAR = MAX-1  
Step 3: If true print Overflow  
Step 4: Else if FRONT = -1 set FRONT = 0  
Step 5: REAR = REAR + 1  
Step 6: Insert element at queue[REAR]  
Step 7: Stop
```

Algorithm for DEQUEUE Operation

Step 1: Start
Step 2: Check if FRONT = -1 OR FRONT > REAR
Step 3: If true print Underflow
Step 4: Else remove queue[FRONT]
Step 5: FRONT = FRONT + 1
Step 6: Stop

C Program for Queue Using Array

```
#include <stdio.h>

#define MAX 5

int queue[MAX];
int front = -1, rear = -1;

void enqueue(int value)
{
    if(rear == MAX-1)
    {
        printf("Queue Overflow\n");
    }
    else
    {
        if(front == -1)
            front = 0;

        rear++;
        queue[rear] = value;
    }
}

void dequeue()
{
    if(front == -1 || front > rear)
    {
        printf("Queue Underflow\n");
    }
    else
    {
        printf("Deleted Element = %d\n", queue[front]);
        front++;
    }
}

void display()
{
    int i;

    for(i=front; i<=rear; i++)
    {
        printf("%d\n", queue[i]);
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
```



```

display();

dequeue();

return 0;
}

```

Output

```

10
20
30
Deleted Element = 10

```

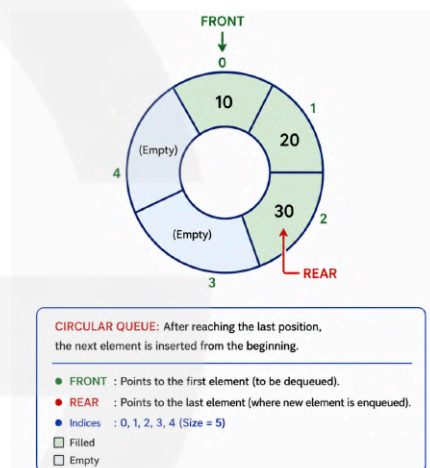
Types of Queues

Different queue types are designed according to processing requirements and memory utilization methods. Important types include circular queue, dequeue, and priority queue.

Circular Queue

Circular queue is modified version of simple queue in which last position is connected back to first position, forming circular structure. Circular queues efficiently utilize memory because freed positions can be reused during insertion operations.

Circular queues are widely used in buffering systems, CPU scheduling, and communication systems.



Advantages of Circular Queue

Circular queues improve memory utilization because unused positions created after deletion can be reused. They reduce memory wastage and increase efficiency of queue operations in communication and operating systems.

Algorithm for Circular Queue Insertion

```

Step 1: Start
Step 2: Check overflow condition
Step 3: If queue empty set FRONT = REAR = 0
Step 4: Else REAR = (REAR + 1) % MAX
Step 5: Insert element
Step 6: Stop

```

Dequeue (Double Ended Queue)

Dequeue is queue structure in which insertion and deletion operations can occur from both front and rear ends. Dequeue provides greater flexibility compared to simple queues and is widely used in scheduling and memory management systems.

Types of Dequeue

Type	Description
Input Restricted Dequeue	Insertion at one end only

Output Restricted Dequeue	Deletion at one end only
---------------------------	--------------------------

Advantages of Dequeue

Dequeue supports flexible insertion and deletion operations from both ends. It improves processing efficiency and is useful in applications requiring bidirectional data manipulation.

Priority Queue

Priority queue is special type of queue in which elements are processed according to priority rather than insertion order. Higher priority elements are removed before lower priority elements regardless of arrival time.

Priority queues are used in operating systems, networking, and scheduling systems.

Types of Priority Queue

Type	Description
Ascending Priority Queue	Smaller priority removed first
Descending Priority Queue	Higher priority removed first

Applications of Priority Queue

Priority queues are used in:

- CPU Scheduling
- Network Routing
- Hospital Emergency Systems
- Printer Scheduling
- Task Management Systems

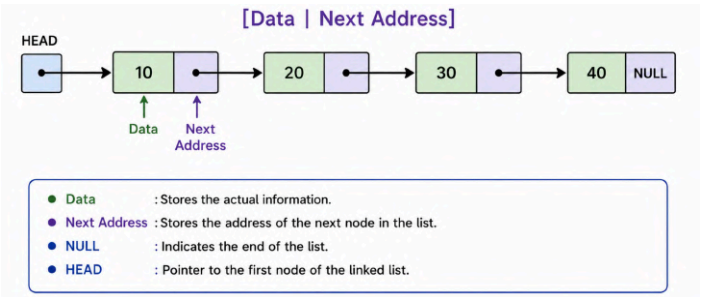
Linked List

A linked list is dynamic linear data structure consisting of nodes connected using pointers. Each node contains:

- Data
- Address of next node

Linked lists provide dynamic memory allocation and efficient insertion/deletion operations.

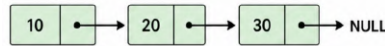
Representation of Linked List



Singly Linked List

Singly linked list is linked structure where each node contains data and pointer to next node only. Traversal occurs in one direction from first node to last node.

Diagram of Singly Linked List



Advantages of Singly Linked List

Singly linked lists provide dynamic memory allocation and efficient insertion/deletion operations. Memory is allocated during runtime, reducing wastage and improving flexibility of data management systems.

Disadvantages of Singly Linked List

Traversal is possible only in forward direction. Additional pointer handling increases implementation complexity, and searching operations are slower compared to arrays because sequential traversal is required.

C Program for Singly Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *next;
};

int main()
{
    struct node *head, *second;

    head = (struct node*)malloc(sizeof(struct node));
    second = (struct node*)malloc(sizeof(struct node));

    head->data = 10;
    head->next = second;

    second->data = 20;
    second->next = NULL;

    printf("%d %d", head->data, second->data);

    return 0;
}
```

Output

10 20

Traversing of Linked List

Traversing means visiting each node of linked list exactly once for processing or displaying data elements sequentially.

Algorithm for Traversing Linked List

Step 1: Start
Step 2: Set TEMP = HEAD
Step 3: Repeat until TEMP = NULL
Step 4: Print TEMP->DATA
Step 5: TEMP = TEMP->NEXT
Step 6: Stop

Searching in Linked List

Searching operation finds desired element within linked list through sequential traversal from first node to last node.

Algorithm for Searching in Linked List

Step 1: Start
Step 2: Set TEMP = HEAD
Step 3: Compare TEMP->DATA with item
Step 4: If found display position
Step 5: Else move TEMP to next node
Step 6: Stop

Insertion in Linked List

Insertion operation adds new node into linked list at beginning, middle, or end position depending upon application requirements.

Algorithm for Insertion at Beginning

Step 1: Start
Step 2: Create NEWNODE
Step 3: NEWNODE->NEXT = HEAD
Step 4: HEAD = NEWNODE
Step 5: Stop

Deletion from Linked List

Deletion operation removes node from linked list while maintaining proper node connections and structure consistency.

Algorithm for Deletion at Beginning

Step 1: Start
Step 2: TEMP = HEAD
Step 3: HEAD = HEAD->NEXT
Step 4: Free TEMP
Step 5: Stop

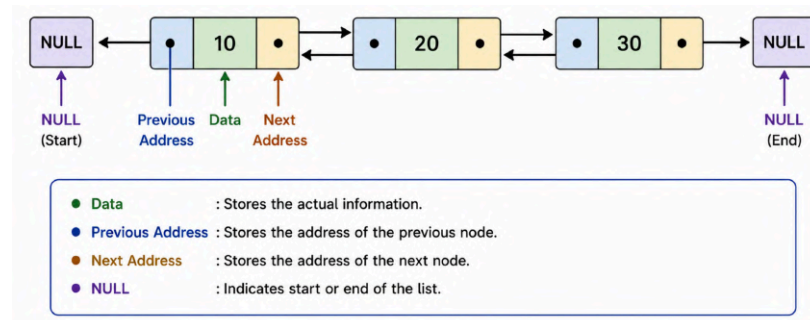
Doubly Linked List

Doubly linked list contains two pointers in each node:

- Previous Pointer
- Next Pointer

This structure supports traversal in both forward and backward directions.

Doubly Linked List Diagram



Advantages of Doubly Linked List

Doubly linked lists support two-way traversal and simplify insertion/deletion operations because previous node information is directly available. They improve navigation efficiency in complex applications.

Disadvantages of Doubly Linked List

Doubly linked lists require additional memory for storing extra pointer. Pointer management becomes more complex compared to singly linked lists.

Header List

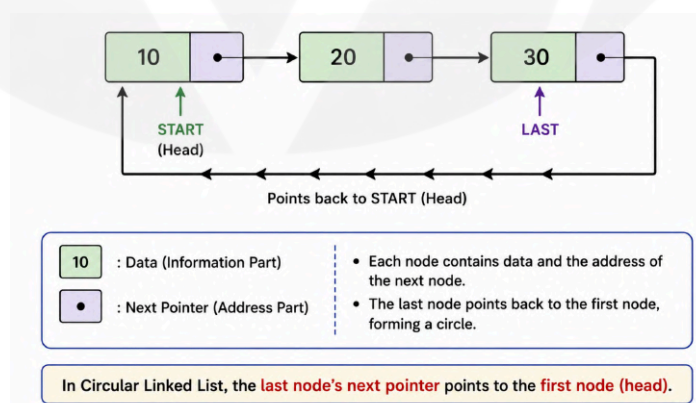
Header list contains special header node placed at beginning of linked list. Header node stores additional information such as node count or starting pointer.

Header lists simplify insertion, deletion, and traversal operations.

Circular Linked List

Circular linked list is linked structure in which last node points back to first node instead of NULL. Circular lists support continuous traversal and are useful in scheduling systems.

Circular Linked List Diagram



Advantages of Circular Linked List

Circular linked lists support continuous processing and efficient traversal without restarting from beginning node. They are widely used in round-robin scheduling and communication systems.

Sorted Linked List

Sorted linked list maintains nodes in ascending or descending order automatically during insertion operations. Sorted lists improve searching efficiency and organized data management.

Advantages of Sorted Linked List

Sorted linked lists maintain ordered structure automatically, simplifying searching and processing operations. They are useful in database systems and priority management applications.

Applications of Linked Lists

Linked lists are widely used in:

- Dynamic Memory Allocation
- Browser History
- Music Playlists
- Polynomial Representation
- Operating System Scheduling
- Stack and Queue Implementation

Comparison Between Queue and Stack

Feature	Stack	Queue
Principle	LIFO	FIFO
Insertion	TOP	REAR
Deletion	TOP	FRONT
Main Use	Recursion	Scheduling

Unit-3 Trees

Basic terminology and definitions. Array and Linked Representation of Binary trees, Traversing Binary trees. Binary Search Trees: Binary Search Tree (BST), Traversal, Insertion and Deletion in BST, and Introduction to balanced BST (AVLTrees).

Introduction to Trees

A tree is non-linear hierarchical data structure consisting of nodes connected through edges. Trees are used to represent hierarchical relationships among data elements. Unlike linear data structures such as arrays and linked lists, trees allow multiple levels of relationships between nodes, making them highly useful for searching, sorting, indexing, and hierarchical data representation.

Trees are widely used in:

- Database Management Systems
- Operating Systems
- Artificial Intelligence
- Compiler Design
- File Systems
- Computer Networks

Trees improve searching efficiency and support dynamic data organization in modern computing systems.

Basic Terminology of Trees

Tree structures contain several important terms that are essential for understanding tree operations and implementations. These terminologies describe relationships among nodes and help programmers perform traversal, insertion, deletion, and searching operations efficiently.

Node

A node is basic element of tree structure that contains data and links to other nodes. Every tree is made up of one or more interconnected nodes arranged hierarchically.

Root Node

Root node is topmost node of tree from which all other nodes originate. Every tree contains only one root node and it does not have any parent node above it.

Parent Node

A parent node is node that has one or more child nodes connected below it. Parent nodes help form hierarchical structure of trees.

Child Node

A child node is node directly connected below another node called parent node. A parent node may have multiple child nodes.

Leaf Node

Leaf node is node that does not contain any child node. Leaf nodes are also called terminal nodes because they appear at end of tree structure.

Edge

Edge is connection between two nodes in tree structure. Edges represent relationships between parent and child nodes.

Level

Level of node represents its position in hierarchy. Root node is generally considered at level 0 or level 1 depending on implementation.

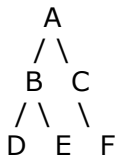
Height of Tree

Height of tree is maximum number of edges from root node to deepest leaf node. Height determines complexity and efficiency of tree operations.

Degree of Node

Degree of node represents number of child nodes connected to that node.

Tree Diagram



Binary Tree

A binary tree is special type of tree in which each node can have maximum two child nodes:

- Left Child
- Right Child

Binary trees are widely used in searching, sorting, expression evaluation, and hierarchical data processing systems because of their efficient organization and traversal methods.

Features of Binary Tree

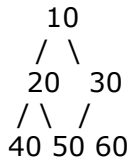
Binary trees provide hierarchical data representation and improve searching efficiency. Each node contains at most two child nodes, simplifying implementation and traversal operations. Binary trees are foundation of advanced structures such as binary search trees and AVL trees.

Features

- Maximum two child nodes

- Hierarchical structure
- Efficient searching support
- Dynamic memory allocation

Binary Tree Diagram



Types of Binary Trees

Type	Description
Full Binary Tree	Every node has 0 or 2 children
Complete Binary Tree	All levels filled except possibly last
Perfect Binary Tree	All internal nodes have two children
Skewed Binary Tree	Nodes arranged in one direction

Array Representation of Binary Tree

In array representation, binary tree nodes are stored sequentially in array. Parent-child relationships are maintained using index positions. Array representation is simple and suitable for complete binary trees.

Representation Rules

If node is stored at index:

i
then:

- Left Child = $2i$
- Right Child = $2i + 1$

Example of Array Representation

Index :	1	2	3	4	5
Data :	A	B	C	D	E

- **Index** : Represents the position of each element in the array.
- **Data** : Represents the actual value stored at that index.
- **Array Size** : Total number of elements = 5
- **Array Notation** : $Arr[1] = A, Arr[2] = B, Arr[3] = C, Arr[4] = D, Arr[5] = E$

Advantages of Array Representation

Array representation provides direct indexing access and simple implementation. It is efficient for complete binary trees where memory wastage is minimal and traversal operations are fast.

Disadvantages of Array Representation

Array representation may waste memory for sparse trees because empty positions remain unused. Dynamic size management is also difficult because arrays have fixed size limitation.

C Program for Array Representation

```
#include <stdio.h>

int main()
{
    char tree[10];

    tree[1] = 'A';
    tree[2] = 'B';
    tree[3] = 'C';

    printf("Root Node = %c\n", tree[1]);
    printf("Left Child = %c\n", tree[2]);
    printf("Right Child = %c\n", tree[3]);

    return 0;
}
```

Output

Root Node = A
Left Child = B
Right Child = C

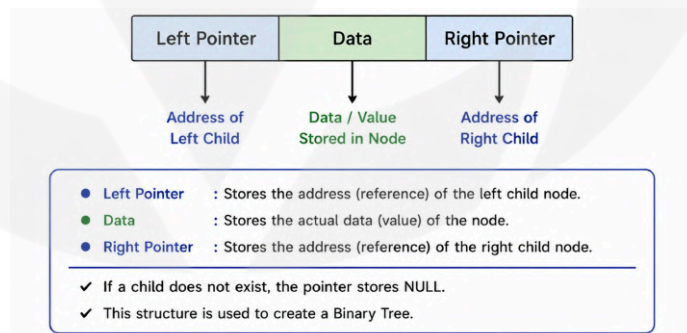
Linked Representation of Binary Tree

Linked representation stores binary tree nodes dynamically using pointers. Each node contains:

- Data
- Pointer to left child
- Pointer to right child

Linked representation is flexible and suitable for dynamic tree structures.

Structure of Binary Tree Node



Advantages of Linked Representation

Linked representation supports dynamic memory allocation and avoids memory wastage. It is suitable for sparse trees and allows efficient insertion and deletion operations.

Disadvantages of Linked Representation

Linked representation requires additional memory for pointers and increases implementation complexity due to pointer management operations.

C Program for Linked Representation

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *left;
    struct node *right;
};

int main()
{
    struct node *root;

    root = (struct node*)malloc(sizeof(struct node));

    root->data = 10;
    root->left = NULL;
    root->right = NULL;

    printf("Root Node = %d", root->data);

    return 0;
}
```

Output

Root Node = 10

Traversing Binary Trees

Traversal means visiting every node of binary tree exactly once in specific order. Traversal operations are important for searching, displaying, sorting, and processing tree elements systematically.

Binary tree traversal is mainly classified into:

- Preorder Traversal
- Inorder Traversal
- Postorder Traversal

Preorder Traversal

In preorder traversal, root node is visited first, followed by left subtree and right subtree.

Order

Root → Left → Right

Algorithm for Preorder Traversal

Step 1: Visit Root Node
Step 2: Traverse Left Subtree
Step 3: Traverse Right Subtree

Example



Preorder Output:

A B C

Inorder Traversal

In inorder traversal, left subtree is visited first, followed by root node and then right subtree. Inorder traversal of BST produces sorted output.

Order

Left → Root → Right

Algorithm for Inorder Traversal

Step 1: Traverse Left Subtree
Step 2: Visit Root Node
Step 3: Traverse Right Subtree

Example

Inorder Output:

B A C

Postorder Traversal

In postorder traversal, left subtree is visited first, then right subtree, and finally root node.

Order

Left → Right → Root

Algorithm for Postorder Traversal

Step 1: Traverse Left Subtree
Step 2: Traverse Right Subtree
Step 3: Visit Root Node

Example

Postorder Output:

B C A

C Program for Tree Traversal

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *left;
    struct node *right;
};
```

```

void inorder(struct node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

int main()
{
    struct node *root;

    root = (struct node*)malloc(sizeof(struct node));

    root->data = 10;
    root->left = NULL;
    root->right = NULL;

    inorder(root);

    return 0;
}

```

Output

10

Binary Search Tree (BST)

Binary Search Tree is special type of binary tree where:

- Left subtree contains smaller values
- Right subtree contains greater values

BST improves searching efficiency because elements are stored in sorted hierarchical order.

Properties of BST

Binary Search Trees maintain ordered structure, allowing efficient insertion, deletion, and searching operations. BST operations generally require less time compared to linear structures such as arrays and linked lists.

Properties

- Left child smaller than parent
- Right child greater than parent
- Recursive ordered structure

BST Diagram

```

    50
   /  \
  30   70

```

/ \ / \
20 40 60 80

Traversal in BST

Traversal methods in BST are same as binary trees. Inorder traversal is especially important because it displays elements in sorted ascending order automatically.

Searching in BST

Searching operation compares target value with current node and moves left or right according to BST properties. This reduces unnecessary comparisons and improves efficiency.

Algorithm for Searching in BST

Step 1: Start from Root
Step 2: Compare item with current node
Step 3: If smaller move left
Step 4: If greater move right
Step 5: Repeat until found or NULL

Insertion in BST

Insertion operation adds new node into BST while maintaining BST properties. New node is inserted at proper position according to comparison rules.

Algorithm for Insertion in BST

Step 1: Start from Root
Step 2: Compare new value
Step 3: Move left if smaller
Step 4: Move right if greater
Step 5: Insert node at empty position

C Program for Insertion in BST

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
    int data;
    struct node *left;
    struct node *right;
};

struct node* create(int value)
{
    struct node *newnode;

    newnode = (struct node*)malloc(sizeof(struct node));

    newnode->data = value;
    newnode->left = NULL;
    newnode->right = NULL;

    return newnode;
}
```

```
}

int main()
{
    struct node *root;

    root = create(50);

    root->left = create(30);
    root->right = create(70);

    printf("Root = %d", root->data);

    return 0;
}
```

Output

Root = 50

Deletion in BST

Deletion operation removes node from BST while preserving BST properties. Deletion becomes more complex because tree structure must remain ordered after removing node.

Cases of BST Deletion

Case	Description
Leaf Node	Direct deletion
One Child	Replace with child
Two Children	Replace with inorder successor

Algorithm for Deletion in BST

Step 1: Search node
Step 2: Identify deletion case
Step 3: Remove node
Step 4: Adjust tree links
Step 5: Maintain BST properties

Introduction to Balanced BST (AVL Trees)

AVL Tree is self-balancing Binary Search Tree in which height difference between left and right subtree is at most: 1

AVL trees maintain balance automatically after insertion and deletion operations.

Features of AVL Trees

AVL trees improve searching efficiency because balanced height reduces traversal time. Rotations are performed automatically to maintain balance and optimize performance.

Features

- Self-balancing structure

- Efficient searching
- Automatic rotations
- Reduced tree height

Balance Factor

Balance Factor is difference between heights of left subtree and right subtree.

Formula

Balance\ Factor = Height(Left\ Subtree) - Height(Right\ Subtree)

For AVL Tree:

$-1 \leq BF \leq 1$

Types of AVL Rotations

Rotation	Description
LL Rotation	Left Left imbalance
RR Rotation	Right Right imbalance
LR Rotation	Left Right imbalance
RL Rotation	Right Left imbalance

Advantages of AVL Trees

AVL trees maintain balanced structure and improve efficiency of insertion, deletion, and searching operations. Balanced height ensures logarithmic time complexity for major tree operations.

Disadvantages of AVL Trees

AVL trees require additional computations for balancing and rotations. Implementation is more complex compared to ordinary binary search trees.

Applications of Trees

Trees are widely used in:

- Database Indexing
- File Systems
- Artificial Intelligence
- Expression Evaluation
- Routing Algorithms
- Compiler Design
- Hierarchical Data Representation

Comparison Between Binary Tree and BST

Feature	Binary Tree	BST
Structure	General hierarchy	Ordered hierarchy
Searching	Slower	Faster
Ordering	No ordering rule	Left < Root < Right
Traversal Output	Unsorted	Sorted in inorder

Unit-4 Searching & Sorting

Searching: Sequential search, binary search, comparison and analysis.

Sorting: Insertion Sort, Bubble Sort, Quick Sort, Two-Way-Merge Sort, Heap Sort, Sorting on Different Keys, Practical consideration for internal Sorting.

Introduction to Searching

Searching is process of finding required element from collection of data items. Searching is one of the most important operations in data structures because large amount of information is stored in computer systems and users frequently need to retrieve specific data efficiently. Efficient searching algorithms reduce execution time and improve software performance.

Searching algorithms are widely used in:

- Database Systems
- Search Engines
- File Systems
- Artificial Intelligence
- Operating Systems
- Banking Applications

The efficiency of searching depends upon:

- Data organization
- Size of dataset
- Searching technique used

Types of Searching

Searching techniques are mainly classified into:

- Sequential Search
- Binary Search

Both techniques differ in efficiency, complexity, and implementation methods.

Sequential Search

Sequential Search is simplest searching technique in which elements are checked one by one sequentially until desired element is found or list ends. It is also called Linear Search. Sequential search is suitable for small and unsorted datasets.

Working of Sequential Search

Sequential search starts from first element and compares each element with target value. If element matches, search stops and position is returned. Otherwise, process continues until end of list.

Advantages of Sequential Search

Sequential search is easy to implement and works on both sorted and unsorted data. It does not require additional memory or preprocessing, making it suitable for small datasets and simple applications.

Disadvantages of Sequential Search

Sequential search is slower for large datasets because every element may need comparison. Its efficiency decreases significantly as size of data increases.

Algorithm for Sequential Search

Step 1: Start
Step 2: Read array and search item
Step 3: Compare item with each array element
Step 4: If found display position
Step 5: Else continue searching
Step 6: If item not found display message
Step 7: Stop

C Program for Sequential Search

```
#include <stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};
    int i, key = 30, found = 0;

    for(i=0; i<5; i++)
    {
        if(arr[i] == key)
        {
            printf("Element found at position %d", i+1);
            found = 1;
            break;
        }
    }

    if(found == 0)
    {
        printf("Element not found");
    }

    return 0;
}
```

Output

Element found at position 3

Time Complexity of Sequential Search

Case	Complexity
Best Case	O(1)
Average Case	O(n)
Worst Case	O(n)

Binary Search

Binary Search is efficient searching technique used on sorted arrays. It repeatedly divides array into two halves and compares middle element with target value. Binary search significantly reduces number of comparisons compared to sequential search.

Binary search follows: Divide and Conquer Technique

Working of Binary Search

Binary search compares target value with middle element:

- If equal, search stops
- If smaller, search left half
- If greater, search right half

This process repeats until element is found or search range becomes empty.

Advantages of Binary Search

Binary search is much faster than sequential search for large datasets. It reduces number of comparisons and provides logarithmic time complexity, making it suitable for large sorted data collections.

Disadvantages of Binary Search

Binary search requires sorted data before searching operation. If data is unsorted, sorting must be performed first, increasing preprocessing time.

Algorithm for Binary Search

Step 1: Start
Step 2: Set LOW = 0 and HIGH = n-1
Step 3: Find MID = (LOW + HIGH)/2
Step 4: Compare item with arr[MID]
Step 5: If equal display position
Step 6: If smaller set HIGH = MID-1
Step 7: If greater set LOW = MID+1
Step 8: Repeat until found or LOW > HIGH
Step 9: Stop

C Program for Binary Search

```
#include <stdio.h>

int main()
{
    int arr[5] = {10,20,30,40,50};
    int low = 0, high = 4, mid, key = 40;

    while(low <= high)
    {
        mid = (low + high)/2;

        if(arr[mid] == key)
```

```

    {
        printf("Element found at position %d", mid+1);
        break;
    }
    else if(arr[mid] < key)
    {
        low = mid + 1;
    }
    else
    {
        high = mid - 1;
    }
}

return 0;
}

```

Output

Element found at position 4

Time Complexity of Binary Search

Case	Complexity
Best Case	$O(1)$
Average Case	$O(\log n)$
Worst Case	$O(\log n)$

Comparison Between Sequential Search and Binary Search

Feature	Sequential Search	Binary Search
Data Requirement	Sorted/Unsorted	Sorted Only
Speed	Slower	Faster
Technique	Sequential Comparison	Divide and Conquer
Complexity	$O(n)$	$O(\log n)$
Suitable For	Small Data	Large Data

Introduction to Sorting

Sorting is process of arranging data elements in specific order such as ascending or descending order. Sorting improves searching efficiency, data organization, and processing performance in software systems.

Sorting algorithms are widely used in:

- Database Systems
- Banking Systems
- Search Engines
- Operating Systems
- Scientific Computing

Types of Sorting

Sorting techniques are mainly classified into:

- Internal Sorting
- External Sorting

Internal sorting occurs when all data fits into main memory.

Insertion Sort

Insertion sort is simple sorting algorithm that builds sorted list one element at time. Each element is inserted into proper position within already sorted portion of array.

Working of Insertion Sort

Insertion sort compares current element with previous elements and shifts larger elements toward right side until correct position is found.

Advantages of Insertion Sort

Insertion sort is simple, stable, and efficient for small datasets. It requires less memory and performs well when data is partially sorted.

Disadvantages of Insertion Sort

Insertion sort becomes inefficient for large datasets because large number of comparisons and shifts are required.

Algorithm for Insertion Sort

Step 1: Start
Step 2: Select second element
Step 3: Compare with previous elements
Step 4: Shift larger elements right
Step 5: Insert element at proper position
Step 6: Repeat for all elements
Step 7: Stop

C Program for Insertion Sort

```
#include <stdio.h>

int main()
{
    int arr[5] = {50,40,30,20,10};
    int i, j, temp;

    for(i=1; i<5; i++)
    {
        temp = arr[i];
        j = i - 1;

        while(j>=0 && arr[j] > temp)
```

```

    {
        arr[j+1] = arr[j];
        j--;
    }

    arr[j+1] = temp;
}

printf("Sorted Array:\n");

for(i=0; i<5; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

Output

Sorted Array:
10 20 30 40 50

Bubble Sort

Bubble sort repeatedly compares adjacent elements and swaps them if they are in wrong order. Larger elements gradually move toward end of array like bubbles rising in water.

Advantages of Bubble Sort

Bubble sort is easy to understand and implement. It is suitable for educational purposes and small datasets.

Disadvantages of Bubble Sort

Bubble sort is inefficient for large datasets because it performs excessive comparisons and swaps.

Algorithm for Bubble Sort

Step 1: Start
 Step 2: Compare adjacent elements
 Step 3: Swap if required
 Step 4: Repeat for all elements
 Step 5: Continue passes until sorted
 Step 6: Stop

C Program for Bubble Sort

```

#include <stdio.h>

int main()
{
    int arr[5] = {50,40,30,20,10};
    int i, j, temp;

    for(i=0; i<5; i++)
    {
        for(j=0; j<4-i; j++)

```

```

    {
        if(arr[j] > arr[j+1])
        {
            temp = arr[j];
            arr[j] = arr[j+1];
            arr[j+1] = temp;
        }
    }
}

printf("Sorted Array:\n");

for(i=0; i<5; i++)
{
    printf("%d ", arr[i]);
}

return 0;
}

```

Output

Sorted Array:
10 20 30 40 50

Quick Sort

Quick sort is efficient divide-and-conquer sorting algorithm. It selects pivot element and partitions array into two parts:

- Elements smaller than pivot
- Elements greater than pivot

Quick sort recursively sorts both partitions.

Advantages of Quick Sort

Quick sort is very fast and efficient for large datasets. It generally provides excellent practical performance and requires less memory.

Disadvantages of Quick Sort

Worst-case performance becomes poor when pivot selection is improper. Recursive implementation increases complexity.

Algorithm for Quick Sort

- Step 1: Select pivot element
- Step 2: Partition array
- Step 3: Place pivot correctly
- Step 4: Recursively sort left partition
- Step 5: Recursively sort right partition
- Step 6: Stop

C Program for Quick Sort

```
#include <stdio.h>
```

```

void quicksort(int arr[], int low, int high)
{
    int i, j, pivot, temp;

    if(low < high)
    {
        pivot = low;
        i = low;
        j = high;

        while(i < j)
        {
            while(arr[i] <= arr[pivot] && i < high)
                i++;

            while(arr[j] > arr[pivot])
                j--;

            if(i < j)
            {
                temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        temp = arr[pivot];
        arr[pivot] = arr[j];
        arr[j] = temp;

        quicksort(arr, low, j-1);
        quicksort(arr, j+1, high);
    }
}

int main()
{
    int arr[5] = {50,40,30,20,10};
    int i;

    quicksort(arr, 0, 4);

    for(i=0; i<5; i++)
    {
        printf("%d ", arr[i]);
    }

    return 0;
}

```

Output

10 20 30 40 50

Two-Way Merge Sort

Merge sort divides array into smaller subarrays, sorts them recursively, and merges sorted subarrays into final sorted array. It follows divide-and-conquer technique.

Advantages of Merge Sort

Merge sort provides stable sorting and guaranteed $O(n \log n)$ performance. It is suitable for large datasets and external sorting applications.

Disadvantages of Merge Sort

Merge sort requires additional memory for merging process and has more implementation complexity.

Algorithm for Merge Sort

Step 1: Divide array into halves
Step 2: Recursively sort halves
Step 3: Merge sorted halves
Step 4: Repeat until fully sorted
Step 5: Stop

Heap Sort

Heap sort uses heap data structure for sorting elements. It first builds max heap and repeatedly removes largest element to create sorted array.

Advantages of Heap Sort

Heap sort provides efficient $O(n \log n)$ performance and does not require additional memory like merge sort.

Disadvantages of Heap Sort

Heap sort implementation is more complex compared to simple sorting algorithms and is generally slower than quick sort in practice.

Algorithm for Heap Sort

Step 1: Build max heap
Step 2: Swap root with last element
Step 3: Reduce heap size
Step 4: Heapify remaining heap
Step 5: Repeat until sorted

Sorting on Different Keys

Sorting on different keys means sorting records according to different attributes or fields. Database systems commonly sort records based on multiple keys such as:

- Name
- Roll Number
- Salary
- Department

Multi-key sorting improves organization and retrieval efficiency.

Example of Different Key Sorting

Name	Age	Salary
Veer	22	30000
Amit	25	25000

Data can be sorted according to:

- Name
- Age
- Salary

Practical Considerations for Internal Sorting

Internal sorting occurs when complete dataset fits into main memory. Several practical factors affect efficiency of internal sorting algorithms.

Important Practical Considerations

Factor	Explanation
Dataset Size	Large datasets need efficient algorithms
Memory Usage	Memory optimization important
Stability	Maintains relative order
Complexity	Lower complexity preferred
Data Nature	Nearly sorted data affects performance

Comparison of Sorting Algorithms

Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

Applications of Searching and Sorting

Searching and sorting algorithms are widely used in:

- Database Systems, Search Engines
- Banking Applications, E-Commerce Systems
- Operating Systems, Artificial Intelligence, Scientific Computing

Unit-5 Graphs

Terminology & Representations, Graphs & Multi-graphs, Directed Graphs, Sequential Representations of Graphs, Adjacency Matrices, Traversal, Connected Component and Spanning Trees, Minimum Cost Spanning Trees.

Introduction to Graphs

A graph is non-linear data structure consisting of collection of vertices (nodes) and edges (connections). Graphs are used for representing relationships between objects in computer systems. Unlike trees, graphs can contain cycles and multiple connections between nodes. Graphs are widely used in computer science for networking, routing, social media analysis, database systems, artificial intelligence, and communication systems.

Graphs are important because they efficiently model real-world problems involving interconnected objects such as:

- Computer Networks
- Road Maps
- Social Networks
- Airline Systems
- Web Links
- Communication Systems

Graphs help programmers solve pathfinding, routing, connectivity, and optimization problems efficiently.

Basic Terminology of Graphs

Graphs contain several important terms that describe relationships and structure of graph systems. Understanding graph terminology is essential for performing graph traversal, searching, spanning tree construction, and optimization operations.

Vertex (Node)

A vertex is fundamental unit of graph representing object or entity. Graph contains one or more vertices connected using edges.

Example

In social network:

- Person = Vertex

Edge

An edge is connection between two vertices in graph. Edges represent relationships between nodes.

Example

Friendship connection in social network.

Adjacent Vertices

Two vertices are called adjacent if edge exists between them.

Example



A and B are adjacent vertices.

Degree of Vertex

Degree of vertex is number of edges connected to that vertex.

Example

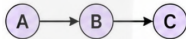


A connected to 3 edges
 $\text{Degree}(A) = 3$

Path

Path is sequence of vertices connected through edges.

Example



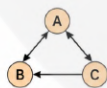
$A \rightarrow B \rightarrow C$

Cycle

Cycle is closed path where starting and ending vertex are same.

Example

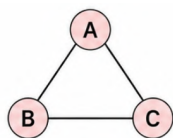
$A \rightarrow B \rightarrow C \rightarrow A$



Connected Graph

Graph is connected if path exists between every pair of vertices.

Graph Representation



Types of Graphs

Graphs are classified according to direction, connectivity, and number of edges between vertices.

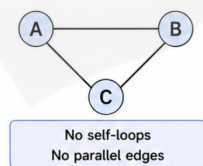
Major Types

- Simple Graph
- Multi-graph
- Directed Graph
- Undirected Graph
- Weighted Graph

Simple Graph

Simple graph contains vertices connected by single edge and does not contain self-loops or multiple edges between same pair of vertices.

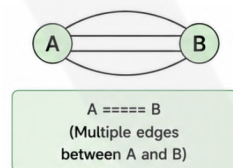
Example of Simple Graph



Multi-graphs

Multi-graph is graph that allows multiple edges between same pair of vertices. Multi-graphs are useful in communication networks and transportation systems where multiple connections may exist between nodes.

Example of Multi-graph



Advantages of Multi-graphs

Multi-graphs represent multiple relationships efficiently and are suitable for transportation systems, communication systems, and parallel network connections.

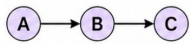
Directed Graphs

Directed graph, also called digraph, is graph in which edges have specific direction. Direction indicates one-way relationship between vertices.

Directed graphs are widely used in:

- Web Page Links
- Network Routing
- Traffic Systems
- Task Scheduling

Directed Graph Representation



Features of Directed Graphs

Directed graphs represent one-way communication and dependency relationships. They support pathfinding and routing operations in networking and scheduling systems.

Features

- Directional edges
- Ordered relationships
- One-way communication

Sequential Representation of Graphs

Sequential representation stores graph using arrays or matrices. This method is simple and suitable for graphs with fixed number of vertices.

Sequential representations mainly include:

- Adjacency Matrix
- Incidence Matrix

Advantages of Sequential Representation

Sequential representation provides direct access to edges and simplifies graph implementation. Matrix-based structures support fast connectivity checking operations.

Disadvantages of Sequential Representation

Sequential representation may waste memory when graph contains fewer edges because many matrix positions remain unused.

Adjacency Matrix

Adjacency matrix is two-dimensional array used for representing graph connections. Rows and columns represent vertices while matrix elements indicate whether edge exists between vertices.

If edge exists:

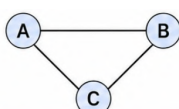
$$\text{Matrix}[i][j] = 1$$

Otherwise:

$$\text{Matrix}[i][j] = 0$$

Example of Adjacency Matrix

Graph:



Adjacency Matrix:

	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

Advantages of Adjacency Matrix

Adjacency matrices provide fast edge lookup operations and simple implementation. They are suitable for dense graphs where large number of edges exist.

Disadvantages of Adjacency Matrix

Adjacency matrices require large memory for sparse graphs because unused matrix positions still consume memory resources.

C Program for Adjacency Matrix

```
#include <stdio.h>

int main()
{
    int graph[3][3] =
    {
        {0,1,1},
        {1,0,1},
        {1,1,0}
    };

    int i, j;

    printf("Adjacency Matrix:\n");

    for(i=0; i<3; i++)
    {
        for(j=0; j<3; j++)
        {
            printf("%d ", graph[i][j]);
        }

        printf("\n");
    }

    return 0;
}
```

Output

Adjacency Matrix:

```
0 1 1
1 0 1
1 1 0
```

Graph Traversal

Traversal means visiting every vertex of graph exactly once in systematic order. Graph traversal is essential for searching, pathfinding, connectivity checking, and network analysis operations.

Graph traversal methods are:

- Breadth First Search (BFS)
- Depth First Search (DFS)

Breadth First Search (BFS)

BFS traversal visits vertices level by level using queue data structure. It first visits neighboring vertices before moving deeper into graph.

BFS is widely used in:

- Shortest Path Problems
- Networking Systems
- Social Networks

BFS Algorithm

Step 1: Start from source vertex
Step 2: Mark vertex visited
Step 3: Insert vertex into queue
Step 4: Remove vertex from queue
Step 5: Visit adjacent unvisited vertices
Step 6: Repeat until queue empty

BFS Example

A → B → C

BFS Output:

A B C

Advantages of BFS

BFS finds shortest path in unweighted graphs and systematically explores neighboring vertices level by level.

Disadvantages of BFS

BFS requires additional memory for queue and may consume more memory in large graphs.

Depth First Search (DFS)

DFS traversal explores graph deeply before backtracking. It uses stack or recursion for traversal operations.

DFS is widely used in:

- Pathfinding
- Topological Sorting
- Cycle Detection

DFS Algorithm

Step 1: Start from source vertex

Step 2: Mark vertex visited

Step 3: Visit adjacent unvisited vertex

Step 4: Repeat recursively

Step 5: Backtrack when no adjacent vertex exists

DFS Example

$A \rightarrow B \rightarrow C$

DFS Output:

A B C

Advantages of DFS

DFS requires less memory compared to BFS and efficiently explores deep graph paths.

Disadvantages of DFS

DFS may not provide shortest path and can become trapped in deep paths without proper implementation.

Connected Component

Connected component is subgraph in which every pair of vertices is connected through some path. Connected components help analyze connectivity structure of graph systems.

Graphs may contain:

- Single Connected Component
- Multiple Connected Components

Example of Connected Components



This graph contains: 2 Connected Components

Applications of Connected Components

Connected components are used in:

- Social Network Analysis

- Network Failure Detection
- Image Processing
- Clustering Systems

Spanning Trees

Introduction

A spanning tree is subgraph that connects all vertices of graph without forming cycles. Spanning tree contains:

$n - 1$ edges
for graph with:

n vertices

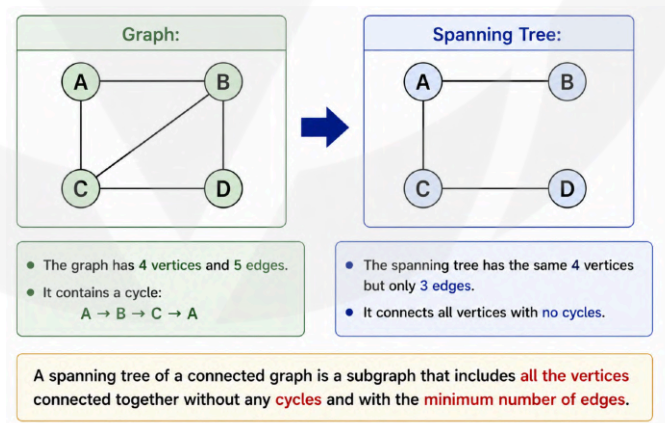
Properties of Spanning Tree

Spanning trees connect all vertices while minimizing number of edges. They remove cycles and maintain graph connectivity efficiently.

Properties

- No cycles
- Connected structure
- Contains all vertices
- Minimum edges required

Example of Spanning Tree



Applications of Spanning Trees

Spanning trees are widely used in:

- Network Design
- Telephone Systems
- Electrical Networks

- Routing Systems

Minimum Cost Spanning Trees (MST)

Minimum Cost Spanning Tree is spanning tree with minimum total edge cost among all possible spanning trees. MST is important optimization technique used in networking and communication systems.

MST helps reduce:

- Communication cost
- Wiring cost
- Network setup cost

Properties of MST

Minimum Cost Spanning Trees connect all vertices with minimum possible edge weights while avoiding cycles.

Features

- Minimum total cost
- Connected structure
- No cycles
- Efficient communication paths

Kruskal's Algorithm

Kruskal's Algorithm constructs MST by selecting edges in increasing order of weights while avoiding cycles.

Algorithm for Kruskal's MST

Step 1: Sort edges by weight
Step 2: Select smallest edge
Step 3: Check cycle formation
Step 4: Include edge if no cycle
Step 5: Repeat until MST complete

Prim's Algorithm

Prim's Algorithm constructs MST by starting from one vertex and repeatedly selecting minimum weighted adjacent edge.

Algorithm for Prim's MST

Step 1: Start from source vertex
Step 2: Select minimum weighted edge
Step 3: Add new vertex to tree
Step 4: Repeat until all vertices included

Advantages of MST

MST reduces communication cost and improves efficiency of network design systems. It ensures minimum resource usage while maintaining complete connectivity.

Applications of MST

Minimum Cost Spanning Trees are used in:

- Computer Networks
- Telephone Networks
- Electrical Grid Systems
- Transportation Systems
- Water Supply Networks

Comparison Between BFS and DFS

Feature	BFS	DFS
Data Structure	Queue	Stack/Recursion
Traversal Method	Level Wise	Depth Wise
Shortest Path	Yes	No
Memory Usage	More	Less

Comparison Between Trees and Graphs

Feature	Tree	Graph
Cycles	Not Allowed	Allowed
Root Node	Present	Not Necessary
Connectivity	Always Connected	May be disconnected
Hierarchy	Hierarchical	General Network